

Homework6

姓名：邢添琨

学号：2024202862

Problem 1

答：首先计算每次 malloc 实际需要分配的 block size。

因为每个 allocated block 都需要 header 和 footer，并且分配器保持 double word alignment，所以：

```
malloc(4):  
payload = 4 bytes  
header + footer = 8 bytes  
总大小 = 12 bytes  
按 8 bytes 对齐后 共 16 bytes
```

```
malloc(10):  
payload = 10 bytes  
header + footer = 8 bytes  
总大小 = 18 bytes  
按 8 bytes 对齐后 共 24 bytes
```

```
malloc(16):  
payload = 16 bytes  
header + footer = 8 bytes  
总大小 = 24 bytes  
已经满足 8 bytes 对齐 共 24 bytes
```

因此：

```
P3 需要 16 bytes  
P4 需要 24 bytes  
P5 需要 24 bytes
```

初始 heap 状态可以表示为：

[F32] [A16:P1] [F16] [A16:P2] [F24]

其中：F32 表示大小为 32 bytes 的 free block，A16:P1 表示大小为 16 bytes、由 P1 指向的 allocated block

Problem 2

First Fit

初始 heap：

[F32] [A16:P1] [F16] [A16:P2] [F24]

执行第 1 条语句后

```
P3 = malloc(4);
```

P3 需要 16 bytes。第一个 free block 是 F32，可以放下。

因此将 F32 拆分为：

A16 + F16

执行后 heap 为：

[A16:P3] [F16] [A16:P1] [F16] [A16:P2] [F24]

执行第 2 条语句后

```
P4 = malloc(10);
```

P4 需要 24 bytes。

First fit 从头开始扫描：第一个 F16 太小，不能放下 24 bytes。第二个 F16 太小，不能放下 24 bytes。最后的 F24 正好可以放下

所以 P4 被分配到最后的 F24 中。

执行后 heap 为：

[A16:P3] [F16] [A16:P1] [F16] [A16:P2] [A24:P4]

执行第 3 条语句后

```
free(P1);
```

释放 P1。P1 对应的 block 大小为 16 bytes。

释放后，P1 左右两边都是 free block：

[F16] [A16:P1] [F16]

释放 P1 后，三个相邻的 free block 会合并：

$F16 + F16 + F16 = F48$

执行后 heap 为：

[A16:P3] [F48] [A16:P2] [A24:P4]

执行第 4 条语句后

```
P5 = malloc(16);
```

P5 需要 24 bytes。第一个可用的 free block 是 F48，可以放下 24 bytes。

因此将 F48 拆分为：

A24 + F24

执行后 heap 为：

[A16:P3] [A24:P5] [F24] [A16:P2] [A24:P4]

Best Fit

初始 heap：

[F32] [A16:P1] [F16] [A16:P2] [F24]

执行第 1 条语句后

```
P3 = malloc(4);
```

P3 需要 16 bytes。Best fit 会选择所有能放下 16 bytes 的 free block 中最小的那个。当前 free block 有：

F32

F16

F24

其中最合适的是 F16，正好放下 16 bytes。

执行后 heap 为：

[F32] [A16:P1] [A16:P3] [A16:P2] [F24]

执行第 2 条语句后

```
P4 = malloc(10);
```

P4 需要 24 bytes。

当前 free block 有：

F32

F24

Best fit 选择最小且能放下 24 bytes 的 free block，也就是 F24。

执行后 heap 为：

[F32] [A16:P1] [A16:P3] [A16:P2] [A24:P4]

执行第 3 条语句后

```
free(P1);
```

释放 P1。P1 对应的 block 大小为 16 bytes。

P1 左边是 F32，右边是已经分配给 P3 的 allocated block。

所以释放后，P1 会和左边的 F32 合并：

$F32 + F16 = F48$

执行后 heap 为：

[F48] [A16:P3] [A16:P2] [A24:P4]

执行第 4 条语句后

```
P5 = malloc(16);
```

P5 需要 24 bytes。

当前只有一个 free block :

F48

Best fit 选择 F48 , 并将其拆分为 :

A24 + F24

执行后 heap 为 :

[A24:P5] [F24] [A16:P3] [A16:P2] [A24:P4]

峰值利用率

heap 总大小不变 :

$32 + 16 + 16 + 16 + 24 = 104$ bytes

下面计算每一步之后的 allocated payload 总大小。

初始状态 :

P1 payload = 8

P2 payload = 8

total payload = 16 bytes

执行第 1 条语句后 :

P1 payload = 8

P2 payload = 8

P3 payload = 4

total payload = 20 bytes

执行第 2 条语句后 :

P1 payload = 8
P2 payload = 8
P3 payload = 4
P4 payload = 10

total payload = 30 bytes

执行第 3 条语句后：

P2 payload = 8
P3 payload = 4
P4 payload = 10

total payload = 22 bytes

执行第 4 条语句后：

P2 payload = 8
P3 payload = 4
P4 payload = 10
P5 payload = 16

total payload = 38 bytes

所以整个过程中最大的 allocated payload 总大小是：

38 bytes

因此峰值利用率为：

$38 / 104 = 0.3654 = 36.54\%$

First fit 和 best fit 在本题中的峰值利用率相同。